

Architecting a zero-DBU, CDC-native reverse ETL pipeline

How a Dagster component turns Delta table commits into Debezium events and streams them downstream, without touching Databricks compute.

TARGET_AUDIENCE: [Data Eng, Data Arch, Tech Lead] // CLEARANCE: Open

Delta changes, two targets, zero DBUs



TL;DR EXECUTION SUMMARY

Zero-DBU reads. delta-rs reads Delta files straight from storage, no warehouse.

Real change events. Snapshot diffs per commit become Debezium c, u and d.

One topic, two targets. Postgres and RisingWave, run by Dagster.

delta-rs: a Delta reader with no JVM



</> What it is

A Rust implementation of the Delta Lake protocol with Python bindings. It reads the table straight from storage with a service principal.

🔍 What we use

The table version, any past version, each commit's file actions, the history for timestamps, and deletion-vector masks.

⚠ What we found

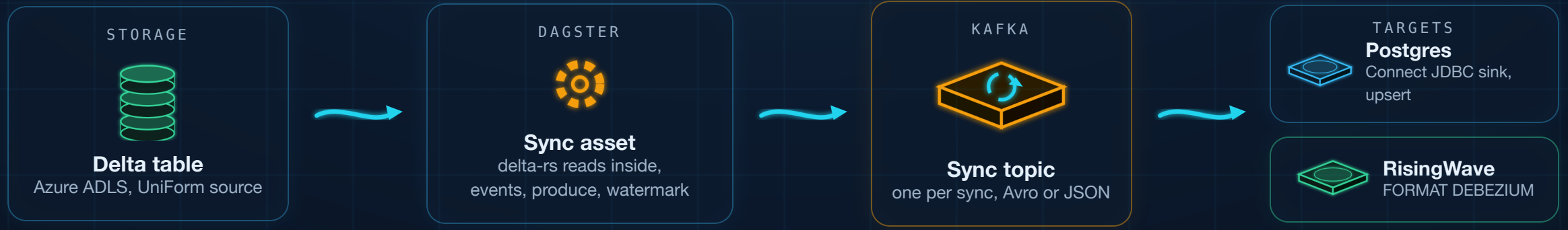
It cannot commit to warehouse-written v7 tables, and column-mapped sources need a direct parquet fallback. Both are handled in code.

Why no off-the-shelf tool fits

Evaluation criteria	drt	dlt	Airbyte	Custom Dagster component
Kafka destination	**No,** by design	**No,** custom module	**Yes**	**Yes,** writes to Kafka
Reads Delta without a warehouse	**Yes,** in newer releases	**No,** SQL warehouse	**No,** JDBC warehouse	**Yes,** delta-rs, zero DBU
Change events (c, u, d)	**No,** no CDC extraction	**No,** row-level merge	**No,** batch snapshots	**Yes,** native Debezium
Fits our Dagster stack	Sensors available	dagster-dlt assets	**No,** own orchestration	**Yes,** assets and checks

⚠ **SaaS tools (Hightouch, Census, RudderStack) were excluded on cost,** and would still need a warehouse or storage credential to read Databricks.

From Delta storage to downstream consumers



Watermark: one appended row per sync in `reverse_etl_sync_state` — postgres by default, or a local DuckDB file (`state_backend`). Rows carry the guard columns `source_fqn` and `label`. Nothing about sync state lives in Databricks.

One YAML file generates the whole sync

```yaml
type: orchestration.components.ReverseEtlCdfSync
name: delta
catalog: de_dev
schema_name: sr_poc_external
source_table: reverse_etl_delta_source
key_column: id
state_backend: postgres   # required, no default
encoding: avro            # optional: json | avro
```



</> **Names are derived, not retyped.** Everything `reverse_etl_<name>_<...>` follows the label, so parallel syncs never collide. **A rename now fails loudly — the watermark guard — it never silently restarts;** the explicit `sync_name` override keeps the watermark across a rename.

Anatomy of one sync run

1. **Pre-check**
   Same version as the watermark? End as a no-op.
2. **Watermark**
   Freshest state row. None means first run.
3. **Read changes**
   Rebuild each later commit with delta-rs.
4. **Envelopes**
   Group rows into Debezium before, after, op.
5. **Add columns**
   Alter RisingWave before producing.
6. **Produce**
   Avro or JSON. Delivery is verified.
7. **Drops, renames**
   Only when the recorded schema proves it.
8. **Commit**
   Append the watermark, last, after produce.

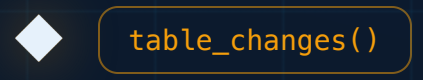
💡 **Crucial ordering.** The watermark moves only after produce is verified. A failed run replays the same versions.

🕑 **Cheap no-op.** Equal versions end the run in about 8 s. The check has a 5 s watchdog and fails open.

📦 **Steps 5 and 7 are the demo's propagation hooks** around the sync's extension points. The released component runs 1–4, 6 and 8 only — target schema work is target-side.

Reading Delta changes without Databricks compute

Databricks SQL path (baseline)



- Needs Change Data Feed enabled on the source table. The Databricks notebook option relies on the same table_changes().
- Every sync meters DBUs: 4.97 on Oct 12 and 0.45 on Oct 13 while this path ran.
- With deletion vectors on, a 1-row delete made CDF emit 300,000 change rows for a 100k-row table.

delta-rs path (zero DBU)



- No Change Data Feed needed: it works on any Delta table.
- Append-only commit: read only the new files.
- Rewrite commit: diff only the removed and added file pairs.
- Deletion vectors: apply each file's keep-mask at that version.

Diffing per version is required: a key deleted in one commit and reinserted in the next must emit both events.

The watermark is one row, written last

reverse_etl_sync_state

reverse_etl_avro	v(n−2)
reverse_etl_avro	v(n−1)
reverse_etl_avro	v(n) **freshest**

+ one appended row per sync · source_fqn, label guard columns

Append-only. Reads take the freshest row per sync. This replaced a MERGE that cost about 2 s per real sync.

Resume at +1. The last version is already processed, so reads start at last_version + 1.

Written last. Only after produce succeeded and the version moved. A failed run replays.

💡 The state used to be a Delta table. Moving it to Postgres on 2026-10-22 left a sync with zero Databricks warehouse touches.

One update, one Debezium envelope




```
{
  before: {...},
  after: {...},
  op: "u",
  source: {...}
}
```

What the key has to be. Unique and non-null, and nothing enforces it, since Databricks doesn't enforce primary keys. In the demo it's the rid identity column. On a real table you pick the primary key yourself.

Where metadata goes. Row columns go in before and after. The commit version and time go in a separate source block, as Debezium consumers expect.

At-least-once, never a silent loss

```
producer.produce(...,
    on_delivery=collect_errors)
undelivered = producer.flush(
    timeout=120)
if undelivered or errors:
    raise RuntimeError(...)
```

produce() only queues the message. It returns before Kafka confirms. A failed send is reported later, in the background.

The flush waits and checks. The run waits up to 120 s for Kafka to confirm every message. Any failure stops it before the watermark moves.

The next sync sends it again. The watermark did not move, so the same versions are replayed. Nothing is skipped.

✓ **Replays are harmless because both targets upsert by key.** Producer tuning: lz4, 50 ms linger, one flush after the last chunk.

Avro cuts the payload by more than 90 percent



1,726 bytes

JSON, schema embedded in every message

12.6x

smaller

→



137 bytes

Avro, schema in the registry

One update event with four columns, computed offline from the same event. Not measured on the topic.

Requirement. A shared staging Apicurio registry. Unchanged schemas are not re-registered.

Constraint. No Avro timestamp form fits both consumers. RisingWave ingests a VARCHAR and generates the TIMESTAMPTZ.

One topic, two independent readers

sync topic → staging brokers



Kafka Connect worker
Debezium JDBC sink



RisingWave
FORMAT DEBEZIUM ENCODE AVRO

Postgres. Reads op d as DELETE and op c or u as UPSERT. Column adds use the sink's own schema evolution.

RisingWave. Applies events straight to a keyed table.

Kafka Connect: open source. Part of Apache Kafka; the Debezium JDBC sink is an open-source plugin (Apache 2.0). **What we run.** One worker container with a 1 GB limit: the Debezium Connect image plus the JDBC sink and Avro converter. **What it needs.** Kafka topics for its own config, offsets and status (local Redpanda), and network access to the brokers and Postgres.

Schema Evolution

Adding a column

1. **Databricks**
   `ALTER TABLE ADD COLUMN`
2. **Dagster asset**
   Reads the schema at the commit
3. **Dagster asset**
   Registers the new Avro version
4. **Dagster asset**
   Alters the RisingWave table
5. **Dagster asset**
   Produces the Kafka events
6. **Postgres sink**
   Adds the column by itself

🔧 **Rename column.** A recorded column id reappears under a new name. Postgres renames natively, then restarts the sink task. RisingWave adds, copies and drops.

🔑 **Drop column.** Only when the recorded schema proves the column is gone. A partial schema read can never trigger a drop.

🔒 **RisingWave constraint.** RisingWave must be altered before the first event carrying the new column is produced, or its values are lost.

📦 **The orchestration in these steps is the demo's hooks** around the sync's extension points — the released sync produces and commits only; targets own their schema.

Releasing to a team: one YAML per concern

```yaml
# the sync — six typed facts, nothing else
type: ReverseEtlCdfSync
name: company
catalog: de_dev
schema_name: their_schema
source_table: dim_company
key_column: Company_ID
state_backend: postgres

# each target is its own YAML — presence = selection
type: ReverseEtlRisingwaveTarget
name: company
source_fqn: de_dev.their_schema.dim_company
key_column: Company_ID
```

Zero Python files adopted. Teams drop YAMLs into their defs folders; the loader merges them.

No Dagster resources. One defs.yaml deploys to staging and production unchanged — values live in the environment (the env contract), never in the YAML.

Names derived by construction. Everything `reverse_etl_<name>_<...>` follows the label; typed fields are only what exists independently.

The watermark guard ships with it. A renamed label fails loudly instead of silently restarting; `sync_name` carries a watermark across a rename.

✓ **The released components import nothing demo-shaped** — pinned by an import-hygiene test, and by a compatibility test that loads them from a plain defs folder.

What the numbers say



Warehouse time per sync

A sync that moves data meters none. The watermark write is a Postgres insert.



One million rows

Ten 100k-row commits with the append fast path. Down from 215 s without it.



No-op sync

The delta-rs pre-check ends an unchanged run. It was about 17 s when a Spark start set the floor.

The Spark path measured 126 to 187 s on the same load, so delta-rs lands in the same range with no Databricks compute behind it.

Where this approach stops

💡 CDF is still the better tool for

- Large, update-heavy sources with many commits per window.
- Sub-minute freshness. Each run here is a batch.
- Vendor-supported semantics instead of owned diff logic.

⚠ Still open in this pipeline

- Partitioned tables without deletion vectors fall back to a full-snapshot diff.
- The first sync is a backfill and reads the full history.
- Each sync walks the whole commit log once for timestamps.

Four bugs found live, now designed out

⚠ Deletion vectors and CDF

A 1-row delete emitted 300,000 change rows. Snapshot diffs are exact however files are rewritten.

🔒 Pin the schema to the commit

A rename between the data read and a live schema read silently nulled a populated column.

▣ Registries dedupe by content

Re-adding a column left an older schema as latest and RisingWave rejected it. Register until yours is latest.

🕑 History lies on pinned handles

delta-rs shifted version labels and gave wrong commit timestamps. Build the map from a current handle.